RENDER V1.3.1-STABLE

RUNES
STABLE · READY

Svelte 5 Tutorial

The Disappearing Framework

A backend engineer's guide to **high-performance reactivity** and compiler-driven UI.

DECLARATIVE

COMPILED

```
let count = $state(0);
const double = $derived(count * 2);
$effect(() => console.log(double));
```

SVELTE 5 REIFIED

16 LESSONS · BUILT BY & FOR BACKEND ENGINEERS

Introduction

This tutorial is an attempt to learn and teach Svelte 5 through the lens of a real-world project. All examples are drawn from **Sparrow**, an open-source webhook delivery platform built using an AI-assisted development workflow.

By exploring how Svelte 5 is used to build Sparrow's management UI, you'll see how runes like $state and $derived solve actual engineering problems—from handling complex delivery filters to visualizing real-time batch progress.

The Sparrow source code is available on GitHub: https://github.com/sarathsp06/sparrow

---

Table of Contents

Audience: Backend engineers who know basic JavaScript/TypeScript and have the Sparrow repo checked out locally. No prior React, Vue, or Svelte experience required.

Approach: Each lesson teaches a Svelte feature with real Sparrow code, then shows what the compiler does with it under the hood. You learn the API and the internals together -- not in separate sections. Conceptual compiled-output illustrations are clearly labeled as such.

Source Code: https://github.com/sarathsp06/sparrow

LESSON 1

Reactive State with $state()

The Problem

In a web app, data changes constantly: a user clicks a button, an API returns results, a timer fires. The UI needs to update to reflect these changes. In vanilla JavaScript, you'd manually find DOM elements and update their text content. Svelte automates this: you declare a variable as reactive, change it, and the UI updates itself.

The Svelte 5 Solution: $state()

The `$state()` rune tells the Svelte compiler to track a variable. When that variable's value changes (via plain assignment), every part of the UI that reads it re-renders automatically.

Source: health/+page.svelte, lines 15-21

```
let loading = $state(true);
let error = $state('');
let healthSummary = $state<HealthSummary | undefined>();
let unhealthyWebhooks = $state<RegisteredWebhook[]>([]);
let degradedWebhooks = $state<RegisteredWebhook[]>([]);
let webhookMetrics = $state(new Map<string, HealthMetrics>());
let namespaceStats = $state<NamespaceStats[]>([]);
```

Line by Line

- `let loading = $state(true)` -- `$state(true)` creates a reactive variable initialized to `true`. While data is loading, the UI shows a skeleton placeholder. When loading completes, `loading = false` triggers the UI to swap to real content.
- `let error = $state('')` -- An empty string means no error. If a fetch fails, `error = 'Something went wrong'` makes an error banner appear.
- `$state<HealthSummary | undefined>()` -- The angle brackets are a generic type parameter (TypeScript). `HealthSummary | undefined` is a union type: the variable is either a HealthSummary object or undefined. Empty parentheses `()` mean no initial value = undefined.
- `$state<RegisteredWebhook[]>([])` -- `RegisteredWebhook[]` means "an array of RegisteredWebhook objects". The `[]` inside parens is the initial value: empty array.
- `$state(new Map<string, HealthMetrics>())` -- A Map is like a dictionary. `Map<string, HealthMetrics>` = keys are strings, values are HealthMetrics objects. Initialized empty.

How Reactivity Triggers

Whether a mutation triggers a UI update depends on the **type** of value wrapped by `$state()`. This is the single most important table in this tutorial:

```
Type            Mutation              Detected?  Why
---             ---                   ---        ---
Primitive       count = 5             Yes        Compiler emits $.set()
(number,                                         which notifies dependents
```

```
 string, bool)

Plain object    obj.name = 'new'       Yes        Proxy set trap fires,
                                                  updates per-property signal

Array           arr.push(item)         Yes        Proxy set trap fires on
                arr[0] = x             Yes        index and length changes

Map             map.set('k', v)        NO         .set() uses internal slots
                                                  that bypass Proxy traps

Set             set.add(v)             NO         Same: .add() bypasses
                                                  Proxy traps

Date            date.setHours(12)      NO         Same: mutation methods
                                                  operate on internal slots
```

For Maps, Sets, and Dates, you must **reassign the entire variable** to trigger an update:

```
// Primitives -- plain assignment:
loading = false;              // $.set() fires

// Objects -- property assignment:
webhook.url = newUrl;         // Proxy set trap fires

// Arrays -- mutation works:
items.push(newItem);          // Proxy set trap fires
items = [...items, newItem];  // Also works (reassignment)

// Maps -- MUST reassign:
webhookMetrics.set(id, m);    // SILENT. No UI update.
webhookMetrics = new Map(     // Correct: reassign
  [...webhookMetrics, [id, m]]
);

// Sets -- MUST reassign:
tags.add("new");              // SILENT. No UI update.
tags = new Set([...tags, "new"]);  // Correct
```

The key insight: `$state()` is a compiler directive, not a regular function. At runtime, the variable holds the plain value (a boolean, a string, an array), not a wrapper object. Svelte's compiler rewrites your code during build to inject change tracking behind the scenes.

Under the Hood: Svelte Is a Compiler

This is the most important thing to understand about Svelte: it is not a runtime library. React, Vue, and Angular ship a runtime that interprets your components in the browser. Svelte is a **compiler** that runs at build time and emits optimized JavaScript -- like the difference between CPython (interpreter) and Go (ahead-of-time compiler). The heavy lifting happens once, so the browser does less.

The Svelte compiler processes each `.svelte` file through three phases, similar to `go build` (parse → type-check → codegen):

```
// sveltejs/svelte — packages/svelte/src/compiler/index.js
export function compile(source, options) {
  // Phase 1: PARSE — source text -> AST      (like go/parser)
  const parsed = _parse(source);

  // Phase 2: ANALYZE — resolve bindings       (like go/types)
  const analysis = analyze_component(
    parsed, source, combined_options
  );

  // Phase 3: TRANSFORM — AST -> optimized JS  (codegen)
  return transform_component(
    analysis, source, combined_options
  );
}
```

Source: sveltejs/svelte -- packages/svelte/src/compiler/index.js

What $state Actually Compiles To

When the compiler sees `$state()`, it replaces it with a **signal** -- a small reactive container object. The rune syntax is just syntactic sugar that disappears at build time:

```
// What you write:
let count = $state(0);

// What the compiler emits (conceptual illustration):
import * as $ from 'svelte/internal/client';
let count = $.state(0);   // creates a signal object

// Template: <p>{count}</p>
// Compiles to:
var p = $.template('<p> </p>');
var text = p.firstChild;
$.render_effect(() => {
  $.set_text(text, $.get(count));  // direct DOM update
});

// No virtual DOM tree. No diffing algorithm.
// One render_effect per dynamic expression.
// Cost = O(changed signals), not O(template size).
```

Contrast this with React, where the entire component function re-executes on every state change, returns a new virtual DOM tree, and React diffs the old and new trees to find what changed. Svelte skips all of that -- the compiler already knows which DOM node to update.

The Signal Object

Under the hood, `$.state(0)` creates a signal -- a plain JavaScript object with a value, a version counter, and a list of who depends on it. If you've worked with the observer pattern (event emitters, pub/sub), this will feel familiar:

```
// sveltejs/svelte — internal/client/reactivity/sources.js
```

```
export function source(v) {
  return {
    f: 0,              // flags (DIRTY, CLEAN, etc.)
    v,                 // current value
    reactions: null,   // effects/deriveds that read this
    equals,            // equality check function
    rv: 0,             // read version
    wv: 0              // write version
  };
}
```

Source: sveltejs/svelte -- packages/svelte/src/internal/client/reactivity/sources.js

When you assign to a `$state` variable, the compiler emits `$.set(signal, newValue)`. This checks equality, updates the value, increments the write version, and walks the signal's reaction list marking each dependent as **DIRTY** -- like a database trigger notifying all subscribed queries that a column changed:

```
// sources.js -- set() (simplified)
export function set(source, value) {
  if (source.equals(value, source.v)) return;

  source.v = value;              // update value
  source.wv++;                   // increment write version
  mark_reactions(source, DIRTY); // notify all dependents
  schedule_effects();            // batch DOM updates
}
```

Deep Reactivity: The Proxy System

When `$state` wraps an object or array, Svelte creates an ES6 Proxy with a **per-property signal Map**. Each property access creates (or reads) a dedicated signal. Changing `obj.name` only re-renders expressions that read `obj.name`, not those that read `obj.age`:

```
// proxy.js (simplified)
export function proxy(value) {
  const sources = new Map();  // property -> signal

  return new Proxy(value, {
    get(target, prop) {
      let s = sources.get(prop);
      if (!s) {
        s = source(target[prop]);
        sources.set(prop, s);
      }
      return get(s);  // registers dependency
    },
    set(target, prop, val) {
      let s = sources.get(prop);
      set(s, proxy(val));  // recursively proxy nested
      return true;
    }
  });
}
```

Why Maps, Sets, and Dates Are Not Reactive

The proxy system works by intercepting property access (`get` trap) and property assignment (`set` trap). For plain objects and arrays, `obj.name = x` triggers the proxy's `set` trap, which updates the per-property signal. But Map, Set, and Date use a different mechanism.

When you call `map.set('key', value)`, JavaScript does this:

```
// What happens at the JS engine level:
//
// 1. map.set  ->  Proxy GET trap fires (reading 'set')
//                 Returns the native Map.prototype.set fn
//
// 2. map.set('key', value)
//    -> Map.prototype.set runs on the RAW Map
//    -> Mutates the Map's [[MapData]] internal slot
//    -> Proxy SET trap NEVER fires
//       (no property was assigned on the proxy object)
//
// The proxy can see you LOOKED UP .set (step 1),
// but cannot see the MUTATION (step 2) because
// it happens inside the engine's C++ implementation,
// not through a property assignment.
//
// Same for Set.add(), Set.delete(), Date.setHours(),
// and all other built-in methods that operate on
// internal slots rather than ordinary properties.
```

Think of it like a Go struct with unexported fields: you can call methods on it, but an external wrapper cannot intercept what those methods do internally. The Proxy is the external wrapper; the Map's internal storage is the unexported field.

Vue 3 handles this differently -- its `reactive()` wraps Map/Set methods (like `.set()`) with custom functions that manually trigger reactivity. Svelte chose not to do this, keeping the proxy implementation simpler and more predictable. The trade-off is that you must remember to reassign Maps and Sets.

$state.raw: Opting Out of Deep Tracking

The proxy system has overhead: every property access goes through a trap, and each property gets its own signal. For large arrays or data you never mutate in place (like API responses you display read-only), use `$state.raw` to skip deep tracking. The variable itself is still reactive -- swapping the whole value triggers updates -- but individual properties are not tracked.

Source: web/src/routes/deliveries/+page.svelte, lines 5, 15, 72-87

```
// Current code (deliveries/+page.svelte, line 15):
let deliveries: WebhookDelivery[] = $state([]);

// The fetch replaces the entire array (lines 77-79):
const res = await deliveryClient
  .listDeliveries(req);
deliveries = res.deliveries || [];

// $state([]) deep-proxies every WebhookDelivery object.
// But we never mutate deliveries[0].status in place --
// we always replace the whole array. Wasted overhead.

// Optimized alternative:
let deliveries = $state.raw<WebhookDelivery[]>([]);
// No per-property signals. Reassigning the array still
// triggers updates. Cheaper for read-only display data.
```

Performance Tiers

Think of reactive state in three tiers, cheapest to most expensive:

- **Tier 1 -- Primitive $state**: `$state(0)`, `$state("")`. A single signal. Cheapest possible.
- **Tier 2 -- $state.raw(obj)**: One signal for the reference. No per-property tracking. Use for API responses and large read-only data.
- **Tier 3 -- $state(obj)**: Proxy + per-property signals. Full deep reactivity. Use when you mutate individual properties.

Gotcha 1: Mutating arrays

If you push to an array with `.push()`, Svelte 5 does detect the mutation (unlike Svelte 4 which required reassignment). However, reassignment is still the clearest pattern: `items = [...items, newItem]`.

Gotcha 2: $state() is not a regular function

You cannot do `const x = condition ? $state(1) : $state(2)`. The compiler must see `$state()` as a direct initializer in a `let` declaration at the top level of a component's script block.

LESSON 2

Computed Values with $derived()

The Problem

You have some state, and you need a value that's calculated from that state. For example: you have an array of webhooks and need a filtered list showing only the active ones. You want to declare the computation once and have it automatically stay in sync.

Simple Form: $derived(expression)

Source: CopyableId.svelte, lines 11-13

```
const display = $derived(
  truncate > 0 && id.length > truncate
    ? id.substring(0, truncate) + '...'
    : id
);
```

This creates a `display` value that automatically recomputes whenever `truncate` or `id` changes. The ternary operator (`? :`) checks: if truncation is enabled and the ID is longer than the limit, show a shortened version; otherwise show the full ID.

Complex Form: $derived.by(() => { ... })

Source: webhooks/+page.svelte, lines 45-57

```
let filteredWebhooks = $derived.by(() => {
  let result = webhooks;
  if (healthFilter) {
    result = result.filter(
      (wh) => wh.health === healthFilter
    );
  }
  if (urlSearch.trim()) {
    const q = urlSearch.toLowerCase();
    result = result.filter((wh) =>
      wh.url.toLowerCase().includes(q) ||
      wh.description?.toLowerCase().includes(q)
    );
  }
  return result;
});
```

Line by Line

- `$derived.by(() => { ... })` -- When the computation needs multiple statements, use `$derived.by()` with a function body.
- `.filter((wh) => wh.health === healthFilter)` -- `.filter()` creates a new array keeping only elements where the callback returns true.

- `wh.description?.toLowerCase()` -- The `?.` is optional chaining. `description` might be undefined. Without `?.`, calling `.toLowerCase()` on undefined would crash.

$derived vs $state: When to Use Which

Use `$state` for data you set directly (API responses, user input). Use `$derived` for data computed from other state. If you find yourself writing code that sets a variable every time another changes, use `$derived`.

Under the Hood: Derived Signals

The compiler transforms `$derived(expr)` into `$.derived(() => expr)`. A derived signal is **lazy** -- it doesn't recompute immediately when a dependency changes. Instead, the runtime marks it as **MAYBE_DIRTY** and only recalculates when something actually reads its value. Think of it as a lazy database view vs. a materialized one.

```
// What happens when a source signal changes:
//
// 1. $.set(source, newValue)
//    -> mark_reactions(source, DIRTY)
//
// 2. For each dependent:
//    - If it's a render_effect -> mark DIRTY (will re-run)
//    - If it's a derived -> mark MAYBE_DIRTY
//      (might not need recalc if other deps cancel out)
//
// 3. Derived only recalculates when $.get() is called.
//    If the source value didn't actually change its
//    equality check, the derived stays clean.
//
// This is push-DIRTY, pull-VALUE:
//   "I know something changed" (push)
//   "Let me check if I need to recompute" (pull)
```

React's `useMemo` requires you to manually list dependencies in an array -- get it wrong and you have stale values or infinite loops. Vue's `computed()` auto-tracks at runtime (closer to Svelte). Svelte's compiler resolves the dependency graph at build time, so there is no dependency array to get wrong and no runtime cost for dependency tracking.

Gotcha 1: Only reactive sources are tracked

`$derived` only tracks variables from `$state`, `$derived`, or `$props`. A plain `let x = 5` is not reactive.

Gotcha 2: Don't mutate inside $derived

A `$derived` computation should be pure. Never modify other `$state` variables inside. That's what `$effect` is for.

LESSON 3

Components & Props with $props()

The Problem

As your UI grows, you need to break it into reusable pieces. A health badge, a copyable ID display, a confirmation dialog. Each piece needs to accept data from its parent. In Svelte 5, components receive data through `$props()`.

Defining Props with TypeScript

Source: CopyableId.svelte, lines 2-8

```
interface Props {
  id: string;
  href?: string;
  truncate?: number;
}

let { id, href, truncate = 8 }: Props = $props();
```

Line by Line

- `interface Props { ... }` -- A TypeScript interface defines the shape of an object.
- `id: string` -- Required prop. Missing it = TypeScript compile error.
- `href?: string` -- The `?` makes this optional. When omitted, the value is undefined.
- `let { id, href, truncate = 8 }: Props = $props()` -- Destructures props. `truncate = 8` is a default value.

Callback Props (Functions as Props)

Source: ConfirmDialog.svelte, lines 1-11

```
interface Props {
  open: boolean;
  title: string;
  message: string;
  confirmLabel?: string;
  cancelLabel?: string;
  variant?: 'danger' | 'warning' | 'info';
  onconfirm: () => void;
  oncancel: () => void;
}
```

- `onconfirm: () => void` -- This prop expects a function. The parent passes a callback. When the user clicks "Confirm", the dialog calls `onconfirm()`.

This is the standard pattern for child-to-parent communication in Svelte 5: the parent passes a function down, the child calls it when something happens.

Under the Hood: What $props Compiles To

The compiler transforms each destructured prop into a signal read via `$.prop()`. When a parent changes a prop value, only the DOM expressions that read *that specific prop* update -- not the entire child component. In React, every prop change re-runs the entire component function.

Source: web/src/lib/components/Pagination.svelte, lines 2-25

```
// Real Sparrow code (Pagination.svelte, lines 2-25):
interface Props {
  currentPage: number;
  totalPages: number;
  totalCount: number;
  pageSize: number;
  onPageChange: (pageNum: number) => void;
  itemLabel?: string;
}

let { currentPage, totalPages, totalCount,
      pageSize, onPageChange,
      itemLabel = 'items' }: Props = $props();

function nextPage() {
  if (currentPage < totalPages) {
    onPageChange(currentPage + 1);
  }
}
function previousPage() {
  if (currentPage > 1) {
    onPageChange(currentPage - 1);
  }
}
```

The compiler transforms this into signal reads. Below is a **conceptual illustration** of the compiled output (not literal Sparrow code):

```
// Conceptual compiled output (illustrative):
// 1. Props become signal reads via $.prop()
let currentPage = $.prop($$props, 'currentPage');
let totalPages  = $.prop($$props, 'totalPages');
let onPageChange = $.prop($$props, 'onPageChange');
let itemLabel = $.prop($$props, 'itemLabel', 8, 'items');

// 2. Template text {currentPage} becomes:
$.render_effect(() => {
  $.set_text(text_node, $.get(currentPage));
});

// 3. disabled={currentPage === 1} becomes:
$.render_effect(() => {
  $.set_attribute(btn, 'disabled',
    $.get(currentPage) === 1);
});

// Each render_effect targets exactly one DOM node.
```

```
// No tree diffing. No re-rendering sibling elements.
```

Gotcha 1: Props are read-only

You cannot reassign a prop inside the child. Props flow one direction: parent to child.

Gotcha 2: Default values only apply when undefined

If parent passes `truncate={0}`, default 8 does NOT apply. 0 is a valid value.

LESSON 4

Conditional Rendering with {#if}

The Problem

Most UI isn't static. You need to show a loading spinner while data loads, an error message when something fails, and actual content when data arrives.

Three-State Pattern: Loading / Error / Content

Source: health/+page.svelte, lines 97-146

```
{#if loading}
  <!-- Skeleton placeholders -->
  <div class="animate-pulse">...</div>
{:else if error}
  <!-- Error message -->
  <div class="text-red-600">{error}</div>
{:else}
  <!-- Actual content -->
  <div>{healthSummary.totalWebhooks}</div>
{/if}
```

How It Works

- `{#if loading}` -- If `loading` is truthy, this branch renders.
- `{:else if error}` -- Check error. An empty string is falsy (no error).
- `{:else}` -- The fallback. Show the real content.
- `{/if}` -- Closes the block.

DOM Destruction vs CSS Hiding

`{#if}` removes elements from the DOM when the condition is false. It doesn't hide them with CSS. Any internal state (scroll position, input values) is lost.

Why destroy instead of hide? Svelte could set `display: none` and keep elements in the DOM, but that has real costs: hidden DOM nodes still consume memory, their event listeners remain active, and any `$effect` inside the branch keeps running. Destroying the branch means zero cost when it's not shown -- no DOM nodes, no event listeners, no effects. For a webhook dashboard with hundreds of conditional sections, this adds up. If you need to preserve internal state across visibility toggles, use CSS (`class:hidden`) instead.

Inline Ternaries in Attributes

```
<span class="{wh.active ? 'bg-green-500' : 'bg-gray-300'}">
  {wh.active ? 'Active' : 'Paused'}
</span>
```

Gotcha 1: Equality checks

JavaScript's == does type coercion. Always use === for strict equality.

Gotcha 2: Empty arrays are truthy

An empty array `[]` is truthy. Use `{#if myArray.length > 0}` instead.

LESSON 5

Rendering Lists with {#each}

The Problem

You have an array of data and need to render each item.

Basic Usage: Table Rows

Source: webhooks/+page.svelte, lines 484-512

```
{#each filteredWebhooks as wh}
  <tr onclick={() => goto(`/webhooks/${wh.webhookId}`)}>
    <td>{wh.url}</td>
    <td>
      <CopyableId id={wh.webhookId}
        href="/webhooks/{wh.webhookId}" />
    </td>
    {#each wh.events.slice(0, 2) as event}
      <span class="badge">{event}</span>
    {/each}
    {#if wh.events.length > 2}
      <span>+{wh.events.length - 2}</span>
    {/if}
  </tr>
{/each}
```

Key Points

- `{#each filteredWebhooks as wh}` -- `wh` is the loop variable holding one webhook per iteration.
- `wh.events.slice(0, 2)` -- Nested `{#each}`. "Show first 2 + overflow count" pattern.

Skeleton Placeholders with Array(n)

Source: health/+page.svelte, lines 104-109

```
{#each Array(4) as _}
  <div class="bg-white animate-pulse">
    <div class="h-8 bg-gray-200 rounded"></div>
  </div>
{/each}
```

- `Array(4)` -- Standard idiom for "repeat N times". `as _` means "I don't use this value."

Gotcha 1: Keyed vs Unkeyed Lists

For reorderable lists, add a key: `{#each items as item (item.id)}`. **Why?** Without a key, Svelte matches DOM nodes by *index*. If you remove item 2 from a list of 5, Svelte updates items 2-4 in place and destroys item 5 -- even though only item 2 was removed. With a key, Svelte matches by identity: it destroys only the removed item's DOM node and leaves the rest untouched. This matters when items have internal state (focused inputs, animations, component instances). Think of it like a database primary key: without it, the DB can't tell which row you mean.

Gotcha 2: Destructuring in the loop

You can destructure: `{#each webhooks as { url, webhookId }}`.

LESSON 6

Inline Constants with {@const}

The Problem

Inside an `{#each}` loop or `{#if}` block, you often need to compute a value. Without `{@const}`, you'd duplicate the expression everywhere.

Map Lookup

Source: health/+page.svelte, line 206

```
{#snippet webhookCard(wh: RegisteredWebhook)}
  {@const metrics = webhookMetrics.get(wh.webhookId)}
  <a href="/webhooks/{wh.webhookId}">
    {#if metrics}
      <span>{(metrics.successRate * 100).toFixed(1)}%</span>
    {/if}
  </a>
{/snippet}
```

- `{@const metrics = webhookMetrics.get(wh.webhookId)}` -- Creates a block-scoped constant. Without it, you'd repeat `.get()` 5+ times.

Computed Value for Rendering

Source: health/+page.svelte, line 228

```
{@const totalErrors = (metrics.clientErrors || 0)
  + (metrics.serverErrors || 0)
  + (metrics.timeoutErrors || 0)
  + (metrics.networkErrors || 0)
  + (metrics.unexpectedStatusErrors || 0)}
```

Function Call Result

Source: deliveries/[deliveryId]/+page.svelte, line 136

```
{#if delivery.errorCategory !== 'success'}
  {@const cat = getCategoryDisplay(delivery.errorCategory)}
  <span class="{cat.bgColor} {cat.color}">
    {cat.label}
  </span>
{/if}
```

Calls the function once, stores the result, then uses `cat.bgColor`, `cat.color`, `cat.label`.

Gotcha 1: Must be at the top of a block

`{@const}` must be first in its block.

Gotcha 2: Truly const

Cannot reassign it. Mutable values belong in the script section as `$state`.

Gotcha 3: No async

`{@const}` is synchronous only. Cannot use `await`.

LESSON 7

Event Handling

The Problem

Users click buttons, type in inputs, press Enter, select dropdowns. In Svelte 5, event handlers are plain HTML attributes -- no special syntax.

Inline Handler

Source: webhooks/+page.svelte, line 487

```
<tr onclick={() => goto(`/webhooks/${wh.webhookId}`)}>
```

- `onclick` -- Standard HTML attribute, lowercase. Different from Svelte 4's `on:click` (deprecated).

Handler with Event Object

Source: webhooks/+page.svelte, line 548

```
<button onclick={(e) => toggleActive(wh, e)}>
```

Named Function Reference

Source: CopyableId.svelte, lines 15-25 & 42

```
async function copyId(e: Event) {
  e.stopPropagation();
  e.preventDefault();
  try {
    await navigator.clipboard.writeText(id);
    copied = true;
    setTimeout(() => { copied = false; }, 1500);
  } catch { /* noop */ }
}

// In template:
<button onclick={copyId}>
```

Keyboard Events

Source: deliveries/+page.svelte, line 267

```
<input
  bind:value={namespaceFilter}
  onkeydown={(e) => e.key === 'Enter' && applyFilters()}
/>
```

Form Submission

Source: events/[eventName]/update/+page.svelte, line 131

```
<form onsubmit={updateEvent}>
  <input bind:value={name} disabled />
  <textarea bind:value={description}></textarea>
  <button type="submit">Update Event</button>
</form>
```

Change Events on Selects

Source: deliveries/+page.svelte, line 276

```
<select bind:value={statusFilter} onchange={applyFilters}>
  <option value="">All</option>
</select>
```

Gotcha 1: handler vs handler()

`onclick={handler}` passes the function. `onclick={handler()}` CALLS it immediately. Almost always a bug.

Gotcha 2: Svelte 5 vs Svelte 4 syntax

Svelte 5 uses `onclick`. Svelte 4 used `on:click`. Both work, but `on:` is deprecated.

LESSON 8

Side Effects with $effect()

The Problem

Sometimes when state changes, you need to do something beyond just updating the UI: start a timer, set up a DOM observer, log analytics. These are side effects. `$effect()` runs a function whenever its reactive dependencies change, and optionally cleans up when dependencies change again or the component is destroyed.

Auto-Dismiss Timer

Source: BatchProgress.svelte, lines 64-74

```
let autoDismissTimer: ReturnType<typeof setTimeout> | undefined;

$effect(() => {
  if (isTerminal && !dismissed) {
    ondone?.();
    autoDismissTimer = setTimeout(() => {
      dismissed = true;
    }, 5000);
  }
  return () => {
    if (autoDismissTimer) clearTimeout(autoDismissTimer);
  };
});
```

Line by Line

- `$effect(() => { ... })` -- Runs after every render where its tracked dependencies change. Here it tracks `isTerminal` (a `$derived`) and `dismissed` (a `$state`).
- `ondone?.()` -- Optional chaining on a function call. If `ondone` was passed as a prop, call it. If not, do nothing.
- `setTimeout(() => { dismissed = true }, 5000)` -- After 5 seconds, auto-dismiss the progress bar.
- `return () => { ... }` -- The cleanup function. Called when (1) the effect re-runs or (2) the component is destroyed. Clears the timer to prevent memory leaks.

IntersectionObserver Pattern

Source: FloatingAction.svelte, lines 7-22

```
let visible = $state(false);

$effect(() => {
  const target = document.querySelector(targetSelector);
  if (!target) return;

  const observer = new IntersectionObserver(
    ([entry]) => {
      visible = !entry.isIntersecting;
    },
    { threshold: 0 }
  );

  observer.observe(target);

  return () => observer.disconnect();
});
```

How It Works

- `IntersectionObserver` -- A browser API that fires a callback when an element enters or exits the viewport.
- `visible = !entry.isIntersecting` -- When the target scrolls out of view, show the floating action button.
- `return () => observer.disconnect()` -- Cleanup: stop observing when the component unmounts.

$effect vs $derived

Use `$derived` for pure computations (input -> output). Use `$effect` for anything that touches the outside world: DOM manipulation, timers, network requests, browser APIs.

Under the Hood: How $effect Tracks Dependencies

The runtime maintains a global variable called `active_reaction`. When an effect runs, it sets itself as the active reaction. Any signal read via `$.get(signal)` during execution registers the active reaction as a dependent. This is how dependencies are tracked automatically -- no manual subscription, no dependency arrays like React's `useEffect([deps])`.

```
// runtime.js (simplified)
let active_reaction = null;  // global: who is running?

export function get(signal) {
  if (active_reaction !== null) {
    // Record: active_reaction depends on this signal
    signal.reactions.push(active_reaction);
  }
  return signal.v;  // return current value
```

```
}

// When an effect runs:
function run_effect(effect) {
  const prev = active_reaction;
  active_reaction = effect;   // set ourselves as active
  effect.fn();                // any $.get() calls register us
  active_reaction = prev;     // restore previous
}
```

Source: sveltejs/svelte -- packages/svelte/src/internal/client/runtime.js

So when the BatchProgress effect runs, it calls `$.get(isTerminal)` and `$.get(dismissed)`. Both signals now have this effect in their `reactions` list. When either signal changes, the effect is marked DIRTY and scheduled to re-run. The old cleanup function runs first, then the effect body executes again.

The compiler transforms `$effect(() => {...})` into `$.user_effect(() => {...})`. The runtime distinguishes between **user effects** (your code, runs after DOM updates) and **render effects** (compiler-generated, updates the DOM). You never create render effects directly -- the compiler creates them from your template expressions.

Gotcha 1: Don't use $effect for derived state

Never write `$effect(() => { count = items.length })`. Use `$derived` instead. **Why?** Effects that set `$state` variables create a hidden dependency cycle: the effect reads a signal (triggering registration), sets another signal (marking dependents dirty), which may trigger more effects. In the worst case, this causes an infinite loop. Even when it works, it creates an unnecessary intermediate state: the UI renders once with the stale value, then again with the updated one (two renders instead of one). `$derived` avoids both problems -- it computes the value lazily, inline, with no intermediate state.

Gotcha 2: Cleanup is critical

If your effect creates timers, observers, or event listeners, always return a cleanup function. Forgetting cleanup causes memory leaks.

Gotcha 3: $effect runs after render

`$effect` runs asynchronously after the DOM updates. If you need to read DOM measurements, `$effect` is the right place.

LESSON 9

Snippets & Render

The Problem

You need to reuse a chunk of HTML template within a component, or pass template content from a parent to a child. In Svelte 4 this was done with slots. Svelte 5 replaces slots with snippets -- explicit, typed, and more powerful.

Declaring a Snippet in a Template

Source: health/+page.svelte, lines 205-259

```
{#snippet webhookCard(wh: RegisteredWebhook)}
  {@const metrics = webhookMetrics.get(wh.webhookId)}
  <a href="/webhooks/{wh.webhookId}">
    <span>{wh.description || 'Webhook'}</span>
    <HealthBadge health={wh.health} size="sm" />
    {#if metrics}
      <span>{(metrics.successRate * 100).toFixed(1)}%</span>
    {/if}
  </a>
{/snippet}

<!-- Used twice: -->
{#each unhealthyWebhooks as wh}
  {@render webhookCard(wh)}
{/each}
{#each degradedWebhooks as wh}
  {@render webhookCard(wh)}
{/each}
```

How It Works

- `{#snippet webhookCard(wh: RegisteredWebhook)}` -- Declares a reusable template block. Think of it like a function that returns HTML.
- `{@render webhookCard(wh)}` -- Calls the snippet, passing the current webhook. Rendered inline at the call site.
- Defined once, rendered in two loops (unhealthy + degraded). Without snippets, you'd copy the entire card HTML twice.

Snippets as Props (Replacing Slots)

Source: EmptyState.svelte, lines 1-25

```
// EmptyState.svelte:
import type { Snippet } from 'svelte';

interface Props {
  icon?: string;
  title: string;
  description?: string;
  action?: Snippet;  // <-- Snippet type!
}

let { icon, title, description, action } = $props();

// In template:
{#if action}
  {@render action()}
{/if}
```

Passing a Snippet from Parent

Source: SubscriptionManager.svelte, lines 425-433

```
<EmptyState
  icon="link"
  title="No subscriptions yet"
  description="Create subscriptions to define..."
>
  {#snippet action()}
    <button onclick={openCreateModal}>
      Create First Subscription
    </button>
  {/snippet}
</EmptyState>
```

- `action?: Snippet` -- The `Snippet` type represents a renderable template block. Optional because not every empty state needs a button.
- `{#snippet action()}` inside parent -- Defines snippet content inline. The name must match the prop name.

Layout Children Pattern

Source: +layout.svelte, lines 6 & 60

```
let { children } = $props();

<!-- At the bottom of the layout: -->
{@render children?.()}
```

`children` is a built-in snippet prop in SvelteKit that contains the page content. The `?.` guards against undefined during SSR.

Gotcha 1: Snippets replace slots

Svelte 4's `<slot />` is deprecated. Use `{#snippet}` + `{@render}` instead.

Gotcha 2: Snippet scope

Snippets defined inside a component access all variables in the component's scope (closures).

LESSON 10

Two-Way Binding

The Problem

Form inputs need to both display a value and update it when the user types. Svelte's `bind:` directive automates this two-way sync.

bind:value on Inputs

Source: SubscriptionManager.svelte, lines 674-680

```
<input
  type="text"
  bind:value={form.eventName}
  oninput={() => handleEventChange(form.eventName)}
  placeholder="Type event name"
/>
```

- `bind:value={form.eventName}` -- Two-way sync: the input displays the value, and typing updates it automatically.
- The `oninput` handler here is for additional logic (fetching event details), not for the binding itself.

bind:value on Selects

Source: deliveries/+page.svelte, line 276

```
<select bind:value={statusFilter} onchange={applyFilters}>
  <option value="">All</option>
  <option value="delivered">Delivered</option>
  <option value="failed">Failed</option>
</select>
```

$bindable: Two-Way Props

Source: SubscriptionManager.svelte, lines 19-29

```
let {
  webhookId,
  namespace,
  subscriptions = $bindable([]),
  onRefresh,
}: {
  webhookId: string;
  namespace: string;
  subscriptions: EventSubscription[];
  onRefresh?: () => void;
} = $props();
```

How $bindable Works

- `subscriptions = $bindable([])` -- Declares that this prop can be two-way bound. The parent uses `bind:subscriptions={myList}` and changes flow both directions.
- Without `$bindable`, props are read-only (Lesson 3). `$bindable` explicitly opts into two-way communication.
- `$bindable([])` provides a default value if the parent doesn't bind.

When to Use Each Pattern

bind:value -- For native HTML elements (input, select, textarea).
$bindable -- For component props needing two-way flow. Rare -- prefer callback props.
Callback props -- For most child-to-parent communication (Lesson 3). More explicit.

Gotcha 1: bind: is two-way

Changes flow both directions. Setting the variable updates the input; typing updates the variable.

Gotcha 2: $bindable is opt-in

By default, props are read-only. You must explicitly mark a prop as `$bindable`.

LESSON 11

Lifecycle Hooks

The Problem

Components need to do things at specific moments: fetch data when first rendered, clean up timers when removed.

onMount: Fetching Data on Load

Source: health/+page.svelte, line 70

```
import { onMount } from 'svelte';

let loading = $state(true);
let error = $state('');

async function fetchData() {
  loading = true;
  error = '';
  try {
    const res = await healthClient.getHealthSummary({});
    healthSummary = res.summary;
  } catch (e: any) {
    error = formatAPIError(e, 'Failed to load');
  } finally {
    loading = false;
  }
}

onMount(fetchData);
```

How It Works

- `onMount(fetchData)` -- Calls `fetchData` once, after the component is first rendered into the DOM. NOT during SSR.
- The `try/catch/finally` pattern ensures `loading` is always cleared, even on failure.

onDestroy: Cleaning Up

Source: deliveries/+page.svelte, lines 4, 44-46

```
import { onDestroy } from 'svelte';

let pollingTimer: ReturnType<typeof setInterval> | undefined;

onDestroy(() => {
  if (pollingTimer) clearInterval(pollingTimer);
});
```

- `onDestroy` -- Runs when the component is removed from the DOM. Use it to clean up intervals, event listeners, subscriptions.

onMount vs $effect

onMount -- Runs once after first render. Best for initial data fetching.
$effect -- Runs after every render where dependencies change. Best for reactive side effects.

Use `onMount` for one-time setup. Use `$effect` when you need to react to state changes.

Gotcha 1: onMount doesn't run on the server

`onMount` is browser-only. During SSR, the component renders without calling it.

Gotcha 2: Don't forget cleanup

If `onMount` starts an interval, pair it with `onDestroy`. Or use `$effect` with return cleanup.

LESSON 12

Async Data Fetching

The Problem

Every page loads data from a backend API. You need to handle loading state, display errors, and sometimes fetch multiple resources in parallel.

The API Client Layer

Source: lib/services.ts, lines 41-50

```
import { createClient } from '@connectrpc/connect';
import { createConnectTransport } from '@connectrpc/connect-web';
import { WebhookService, EventService } from '../proto/webhook_pb.js';

const transport = createConnectTransport({
  baseUrl: PUBLIC_API_URL || '/',
  interceptors,  // API key injection
});

export const webhookClient = createClient(WebhookService, transport);
export const eventClient = createClient(EventService, transport);
export const deliveryClient = createClient(DeliveryService, transport);
export const healthClient = createClient(HealthService, transport);
```

How It Works

- `createConnectTransport` -- Creates an HTTP transport that speaks the Connect protocol.
- `createClient(WebhookService, transport)` -- Creates a typed client. You call `webhookClient.listWebhooks({...})` and get back typed responses.
- `interceptors` -- Middleware that attaches the `X-API-Key` header when configured.

Parallel Fetching with Promise.all

Source: health/+page.svelte, lines 27-38

```
const [summaryRes, statsRes, unhealthyRes, degradedRes] =
  await Promise.all([
    healthClient.getHealthSummary({}),
    webhookClient.getNamespaceStats({ namespace: '' }),
    healthClient.listWebhooksByHealth({
      health: WebhookHealth.HEALTH_UNHEALTHY,
      pagination: { limit: 20, offset: 0 },
    }),
    healthClient.listWebhooksByHealth({
      health: WebhookHealth.HEALTH_DEGRADED,
      pagination: { limit: 20, offset: 0 },
    }),
  ]);
```

- `Promise.all([...])` -- Runs all four API calls concurrently. Waits for all to complete.

- `const [a, b, c, d] = await ...` -- Array destructuring on the result.

The Loading/Error/Content Pattern

```
let loading = $state(true);
let error = $state('');
let data = $state<DataType | undefined>();

async function fetchData() {
  loading = true;
  error = '';
  try {
    const res = await client.getData({...});
    data = res.data;
  } catch (e: any) {
    error = formatAPIError(e, 'Failed to load');
  } finally {
    loading = false;
  }
}

onMount(fetchData);
```

The template then uses the three-state `{#if}` pattern from Lesson 4.

Error Formatting

Source: lib/utils.ts, lines 217-241

```
function formatAPIError(err: unknown, contextPrefix?: string): string {
  let msg = (err as any)?.message ?? String(err);
  // Strip gRPC code prefix: "[internal] ..."
  msg = msg.replace(/^\[\w+\]\s*/, '');
  if (!contextPrefix) return msg;
  return `${contextPrefix}: ${msg}`;
}
```

Gotcha 1: Always set loading = false

Use `finally { loading = false }`. Without `finally`, an error leaves the page stuck on the skeleton.

Gotcha 2: Promise.all is all-or-nothing

If one call fails, you lose all results. Use `Promise.allSettled()` for partial results.

LESSON 13

Layout & Navigation

The Problem

Every page shares a navigation header. You don't want to copy this HTML into every page. SvelteKit's layout system lets you define shared UI once.

The Layout Component

Source: +layout.svelte, all 60 lines

```
<script lang="ts">
  import { page } from "$app/state";
  import "../app.css";

  let { children } = $props();

  const titles: Record<string, string> = {
    "/webhooks": "Webhooks",
    "/events": "Events",
    "/health": "Health",
  };

  function getTitle(): string {
    const path = page.route.id?.toString() || "/";
    return titles[path] || "";
  }
</script>

<header class="sticky top-0 ...">
  <h2>{getTitle()}</h2>
  <nav>
    <a href="/webhooks">Webhooks</a>
    <a href="/events">Events</a>
    <a href="/deliveries">Deliveries</a>
    <a href="/health">Health</a>
  </nav>
</header>

{@render children?.()}
```

How It Works

- `let { children } = $props()` -- SvelteKit passes a `children` snippet containing the current page's content.
- `{@render children?.()}` -- Renders the page content below the header.
- `page.route.id` -- SvelteKit's reactive page state gives the current route pattern for dynamic titles.
- `import "../app.css"` -- Global CSS (Tailwind) imported in layout, available everywhere.

SvelteKit Navigation: goto()

Source: webhooks/+page.svelte, line 487

```
import { goto } from '$app/navigation';

<tr onclick={() => goto(`/webhooks/${wh.webhookId}`)}>
```

- `goto(url)` -- Programmatic navigation. No full page reload -- SvelteKit does client-side navigation.

Dynamic Routes

```
web/src/routes/
  webhooks/
    +page.svelte          -> /webhooks
    register/+page.svelte -> /webhooks/register
    [webhookId]/+page.svelte -> /webhooks/abc123
  events/
    [eventName]/
      reports/+page.svelte -> /events/user.created/reports
```

- `[webhookId]` -- A dynamic segment. The value is available via `page.params.webhookId`.

svelte:head

Source: health/+page.svelte, lines 73-75

```
<svelte:head>
  <title>Health | Sparrow</title>
</svelte:head>
```

`<svelte:head>` injects elements into the document head. Each page sets its own title.

Gotcha 1: Layout wraps all pages

Everything in `+layout.svelte` appears on every page in that directory (and subdirectories).

Gotcha 2: goto() vs <a>

Prefer `<a href="...">` for regular links. Use `goto()` for programmatic navigation.

LESSON 14

Component Composition

The Problem

Real applications need reusable components that work together: a pagination control, a progress bar, a confirmation dialog. This lesson shows how to design them, combining Lessons 1-13.

Pattern 1: Callback-Driven Component

Source: Pagination.svelte, all 72 lines

```
interface Props {
  currentPage: number;
  totalPages: number;
  totalCount: number;
  pageSize: number;
  onPageChange: (pageNum: number) => void;
  itemLabel?: string;
}

let { currentPage, totalPages, totalCount,
  pageSize, onPageChange, itemLabel = 'items'
}: Props = $props();

// In template:
<button onclick={() => onPageChange(pageNum)}>
  {pageNum}
</button>
```

Pagination is "dumb" -- it displays page numbers and calls onPageChange when clicked. The parent owns the state and data-fetching logic.

Pattern 2: Self-Contained with $effect

Source: BatchProgress.svelte, lines 24-74

```
let { batch, label = 'Batch', oncancel, ondone } = $props();

let dismissed = $state(false);

let progressPercent = $derived(
  batch && batch.total > 0
    ? Math.round(((batch.processed + batch.failed)
        / batch.total) * 100)
    : 0
);

let isTerminal = $derived(
  batch?.status === 'completed' ||
  batch?.status === 'failed'
);

let statusColor = $derived.by(() => {
  switch (batch.status) {
    case 'completed': return 'bg-green-500';
    case 'failed':    return 'bg-red-500';
    case 'processing': return 'bg-blue-500';
    default:           return 'bg-yellow-500';
  }
});

$effect(() => {
  if (isTerminal && !dismissed) {
    ondone?.();
    autoDismissTimer = setTimeout(
      () => { dismissed = true; }, 5000);
  }
  return () => clearTimeout(autoDismissTimer);
});
```

How It Combines Lessons

- **$props()** (Lesson 3) -- Receives batch status and callbacks.
- **$state** (Lesson 1) -- Local `dismissed` state.
- **$derived** (Lesson 2) -- `progressPercent`, `isTerminal`, `statusColor`.
- **$derived.by()** (Lesson 2) -- Switch statement needs block form.
- **$effect** (Lesson 8) -- Auto-dismiss timer with cleanup.
- **Callback props** (Lesson 3) -- `oncancel`, `ondone`.

Pattern 3: Snippet Slot Component

Source: EmptyState.svelte

```
import type { Snippet } from 'svelte';

interface Props {
  icon?: string;
  title: string;
  description?: string;
  action?: Snippet;  // Slot replacement
}

// Template:
{#if action}
  {@render action()}
{/if}
```

EmptyState provides structure (icon, title) and a snippet slot for custom content. The parent decides what action to show; the child decides where.

Design Guidelines

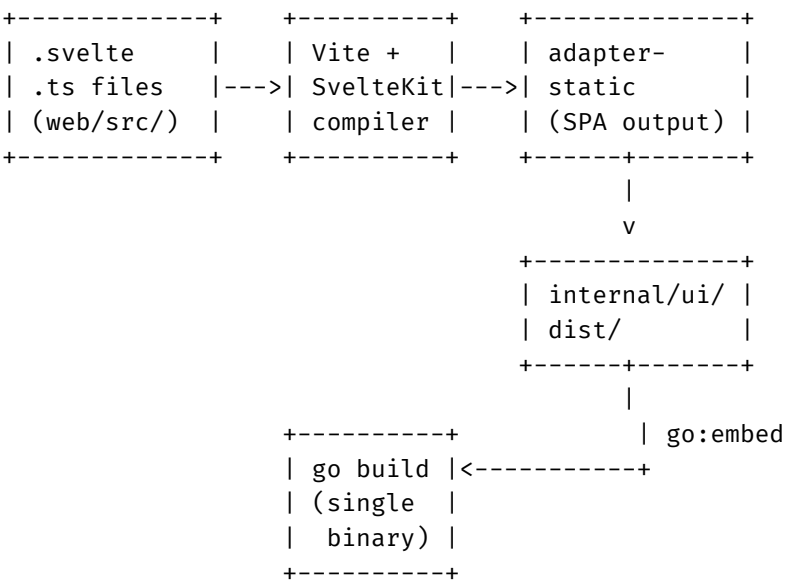
- **Keep components focused** -- Each component has one job.
- **Parent owns state, child reports events** -- Use callback props.
- **Use snippets for flexible content** -- Accept `Snippet` props for custom HTML.
- **Type everything** -- `interface Props` for every component.

LESSON 15

The Build Pipeline

This lesson traces the complete build pipeline for Sparrow's frontend -- from `.svelte` source files to a single Go binary with the UI embedded. Every config file shown is from the actual Sparrow codebase.

Pipeline Overview



Step 1: Vite + SvelteKit Compilation

Vite is the build tool that orchestrates everything. It calls the Svelte compiler as a plugin for each `.svelte` file, then bundles, tree-shakes, and minifies the output.

Source: web/vite.config.ts

```
import { sveltekit } from '@sveltejs/kit/vite';
import tailwindcss from '@tailwindcss/vite';
import { defineConfig } from 'vite';

export default defineConfig({
  plugins: [
    tailwindcss(),   // Tailwind v4 -- CSS at build time
    sveltekit(),     // Svelte compiler + SvelteKit routing
  ],
  resolve: {
    alias: {
      // Force ESM for protobuf (Vite needs ESM)
      '@bufbuild/protobuf': resolve(
        __dirname,
        './node_modules/@bufbuild/protobuf/dist/esm'
      ),
    },
  },
});
```

The plugin order matters: Tailwind runs first (processes `@import 'tailwindcss'` in CSS), then SvelteKit compiles `.svelte` files, and Vite handles bundling and tree-shaking.

Step 2: adapter-static (SPA Mode)

SvelteKit supports multiple deployment targets via adapters. Sparrow uses `adapter-static` which pre-renders to static HTML/JS/CSS files. The `fallback: 'index.html'` option enables SPA mode -- all routes are handled client-side.

Source: web/svelte.config.js

```
import staticAdapter from '@sveltejs/adapter-static';

const config = {
  kit: {
    adapter: staticAdapter({
      pages: '../internal/ui/dist',  // output dir
      assets: '../internal/ui/dist',
      fallback: 'index.html',        // SPA mode
      strict: false  // allow non-prerendered routes
    }),
  }
};
```

The output directory is `../internal/ui/dist` -- this places the built frontend exactly where the Go embed directive expects it.

Step 3: go:embed (Compile into Binary)

Go's `embed` package bakes files into the binary at compile time. Sparrow uses `//go:embed all:dist` to include the entire frontend build output. The `all:` prefix includes dotfiles and hidden files.

Source: internal/ui/embed.go, lines 24-25, 40-88

```
// Real Sparrow code (internal/ui/embed.go):
//go:embed all:dist
var embeddedFS embed.FS

func Handler(logger *slog.Logger,
             config *Config) http.Handler {
  staticFS, _ := fs.Sub(embeddedFS, "dist")
  fileServer := http.FileServer(http.FS(staticFS))
  configScript := buildConfigScript(config)

  return http.HandlerFunc(func(w http.ResponseWriter,
                               r *http.Request) {
    path := strings.TrimPrefix(r.URL.Path, "/")

    // Try to serve static file directly
    if path != "" {
      if f, err := staticFS.Open(path); err == nil {
        _ = f.Close()
        // Immutable assets get 1-year cache
        if strings.HasPrefix(path,
            "_app/immutable/") {
          w.Header().Set("Cache-Control",
            "public, max-age=31536000, immutable")
        }
        fileServer.ServeHTTP(w, r)
        return
      }
    }

    // SPA fallback: serve index.html
    indexBytes, _ := fs.ReadFile(staticFS,
      "index.html")
    html := string(indexBytes)
    // Inject runtime config into </head>
    if configScript != "" {
      html = strings.Replace(html,
        "</head>", configScript+"</head>", 1)
    }
    w.Header().Set("Content-Type",
      "text/html; charset=utf-8")
    w.Write([]byte(html))
  })
}
```

Step 4: Docker Multi-Stage Build

The Dockerfile chains all stages together: Node builds the frontend, Go compiles the binary with the embedded UI, and the final image is distroless (no shell, no package manager, minimal attack surface).

Source: Dockerfile, lines 2-63

```
# Stage 1: Build frontend (Dockerfile, lines 2-23)
FROM node:22-alpine AS frontend
WORKDIR /build/web
COPY web/package.json web/package-lock.json* ./
RUN npm ci --ignore-scripts
COPY web/ .
# Proto files needed by SvelteKit imports:
COPY proto/webhook_pb.js proto/webhook_pb.d.ts \
     /build/proto/
RUN PUBLIC_API_URL=/ npm run build
# Output: /build/internal/ui/dist/

# Stage 2: Build Go binary (lines 25-49)
FROM golang:1.26.1-alpine AS builder
WORKDIR /build
COPY go.mod go.sum ./
RUN go mod download
COPY . .
COPY --from=frontend /build/internal/ui/dist/ \
     /build/internal/ui/dist/
RUN CGO_ENABLED=0 go build \
    -ldflags="-w -s" -trimpath \
    -o server ./cmd/server

# Stage 3: Runtime (lines 51-63)
FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=builder /build/server /app/server
EXPOSE 50051 8080
ENTRYPOINT ["/app/server"]
```

Runtime Config Injection

The Go server injects runtime configuration (like the API key) into `index.html` on every request -- no frontend rebuild needed when configuration changes. The SPA reads `window.__SPARROW_CONFIG__` at startup.

```
// Go server injects into </head>:
<script>window.__SPARROW_CONFIG__={"apiKey":"..."};</script>

// SvelteKit reads it (web/src/lib/services.ts):
const runtimeConfig =
  (typeof window !== 'undefined'
    && window.__SPARROW_CONFIG__) || {};
const apiKey = runtimeConfig.apiKey || '';
```

What Gets Shipped: Bundle Size

Because Svelte compiles away the framework, your bundle contains only the runtime primitives you actually use (signals, effects, template helpers) plus your compiled component code. There is no large framework runtime to download and parse. Approximate industry figures (minified + gzipped):

Framework runtime sizes (approx, min+gzip):

Framework	Runtime shipped	Notes
React 18 + DOM	~42-44 KB	Always included
Vue 3	~33 KB	Always included
Angular 17+	~90-130 KB	Varies by features
Svelte 5	~5-8 KB	Tree-shaken; only used primitives

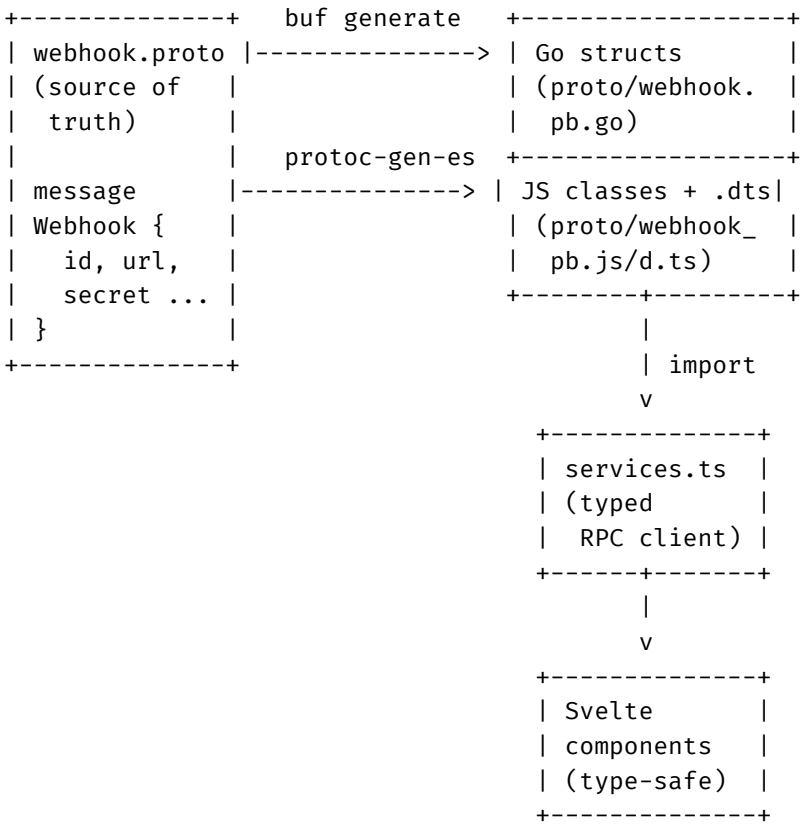
The Svelte runtime is smaller because the compiler inlines only the helpers each component actually uses. React and Vue ship their entire reconciler/VDOM engine regardless of app complexity.

LESSON 16

The Full Stack

This final lesson traces a complete path through Sparrow's stack -- from protobuf definition to compiled Svelte component, embedded in a Go binary, served to the browser. Every technology choice at each layer exists for a concrete reason.

The Journey of a Type



Layer 1: Protobuf -- Single Source of Truth

The `webhook.proto` file defines all types and RPC services once. The `buf generate` command produces Go server code, Go clients, JS/TS clients, and Python clients from this single definition. No type duplication across languages.

Source: buf.gen.yaml

```
# buf.gen.yaml -- generates code for all languages
plugins:
  # Go server (protobuf + gRPC + Connect-RPC)
  - plugin: buf.build/protocolbuffers/go
    out: .
  - plugin: buf.build/connectrpc/go
    out: .

  # Web UI (Connect-RPC / @bufbuild/protobuf v2)
  - name: es
    path: web/node_modules/.bin/protoc-gen-es
    out: .
    opt:
      - target=js+dts  # JS classes + TypeScript types
```

Layer 2: Connect-RPC Transport

Connect-RPC is a protocol-compatible alternative to gRPC-Web that works over standard HTTP. The Svelte frontend creates a typed transport with an interceptor that attaches the API key header to every request.

Source: web/src/lib/services.ts, lines 1-53

```
// Real Sparrow code (services.ts):
import { PUBLIC_API_URL }
  from '$env/static/public';
import { createClient }
  from '@connectrpc/connect';
import { createConnectTransport }
  from '@connectrpc/connect-web';
import type { Interceptor }
  from '@connectrpc/connect';
import {
  WebhookService, EventService,
  SubscriptionService, DeliveryService,
  HealthService
} from '../../../proto/webhook_pb.js';

// Runtime config injected by Go server
const runtimeConfig: SparrowConfig =
  (typeof window !== 'undefined'
    && window.__SPARROW_CONFIG__) || {};
const apiKey = runtimeConfig.apiKey || '';

// Interceptor attaches API key header
const apiKeyInterceptor: Interceptor =
  (next) => async (req) => {
    req.header.set('X-API-Key', apiKey);
    return next(req);
  };

const transport = createConnectTransport({
  baseUrl: PUBLIC_API_URL || '/',
  interceptors: apiKey
    ? [apiKeyInterceptor] : [],
});

// Typed clients (full autocomplete on RPCs)
export const webhookClient =
  createClient(WebhookService, transport);
export const eventClient =
  createClient(EventService, transport);
export const deliveryClient =
  createClient(DeliveryService, transport);
export const healthClient =
  createClient(HealthService, transport);
```

Layer 3: Svelte Components (Type-Safe)

Components call the typed RPC clients and display results. TypeScript ensures the response types match the proto definitions. The Svelte compiler then turns these components into optimized DOM update code -- no virtual DOM, just direct mutations as described in Lessons 1 and 3.

Source: web/src/routes/webhooks/+page.svelte, lines 2-17, 62-98

```
// Real Sparrow code (webhooks/+page.svelte):
import { webhookClient as client,
         healthClient } from '$lib/services';
import { onMount } from 'svelte';
import type { RegisteredWebhook }
  from '../../../../proto/webhook_pb.js';

let webhooks: RegisteredWebhook[] = $state([]);
let loading = $state(true);
let error = $state('');

// Pagination state
let limit = $state(25);
let offset = $state(0);
let totalCount = $state(0);

async function fetchWebhooks() {
  loading = true;
  error = '';
  try {
    const res = await client.listWebhooks({
      namespace: '',
      pagination: { limit, offset },
    });
    // res.webhooks is RegisteredWebhook[]
    // TypeScript enforces field access:
    //   res.webhooks[0].url       OK
    //   res.webhooks[0].bogus     compile error
    webhooks = res.webhooks || [];
    totalCount =
      res.pagination?.totalCount || 0;
  } catch (e: any) {
    error = formatAPIError(e,
      'Failed to load webhooks');
  } finally {
    loading = false;
  }
}

onMount(fetchWebhooks);  // fetch on page load
```

Layer 4: Build + Embed

The SvelteKit build produces static HTML/JS/CSS in `internal/ui/dist/`. The Go compiler's `embed` directive bakes these files into the binary. The result is a single executable with no external dependencies -- no Node runtime, no file server, no CDN required.

```
// The complete build chain:
$ cd web && npm run build
  # Vite -> Svelte compiler -> Tailwind -> tree-shake
  # -> adapter-static -> ../internal/ui/dist/
```

```
$ go build ./cmd/server
  # go:embed all:dist -> embed FS in binary
  # Output: single static binary (~15MB)

$ ./server
  # :8080 -- Connect-RPC API + embedded SPA
  # :50051 -- gRPC (direct)
  # No Node. No nginx. No CDN. One process.
```

Why This Architecture

Each technology at each layer was chosen for a specific reason:

- **Protobuf** -- Single source of truth for types across Go, TypeScript, and Python. Schema evolution without breaking changes.
- **Connect-RPC** -- Works over standard HTTP (no gRPC-Web proxy needed). Browser-compatible. Same proto definitions as gRPC.
- **Svelte 5** -- Compiled to vanilla JS. No runtime framework to ship. ~5-8 KB tree-shaken runtime vs ~44 KB for React. Runes provide type-safe reactivity.
- **adapter-static** -- Pure static files that can be embedded. No SSR server needed. SPA mode for client-side routing.
- **go:embed** -- Single binary deployment. No filesystem dependencies. Copy one file, run it. Works in distroless containers.
- **Distroless** -- No shell, no package manager. ~5MB base image. Minimal attack surface for a self-hosted webhook platform.

The End-to-End Type Safety Chain

The most important property of this architecture is the unbroken type chain. A field added to `webhook.proto` automatically appears in the Go server struct, the TypeScript client, and the Svelte component -- with compile-time errors if any layer gets it wrong. There is no manually-maintained API contract, no JSON schema to keep in sync, no runtime type assertion. The proto file IS the contract, and code generation enforces it everywhere.

Quick Reference Card

Concept	Syntax	Use When
Reactive state	`$state(initialValue)`	Data you set directly
Computed value	`$derived(expr)` `$derived.by(() => { })`	Value calculated from state

Side effects	`$effect(() => {`
	`  ...`
	`  return () => cleanup;`
	`})`
		Timers, DOM, browser APIs

Component props	`let { a, b } = $props()`	Receiving data from parent
Two-way prop	`x = $bindable([])`	Parent reads/writes child state

Conditional	`{#if cond}`
	`{:else if}`
	`{:else}`
	`{/if}`
		Show/hide DOM sections

List rendering	`{#each arr as item}` `{#each arr as x (key)}`	Rendering arrays
Inline constant	`{@const x = expr}`	Avoid repeated expressions

Snippet (define)	`{#snippet name(params)}`
	`  HTML...`
	`{/snippet}`
		Reusable template blocks

Snippet (render)	`{@render name(args)}`	Calling a snippet
Event handler	`onclick={handler}` `onclick={(e) => {}}`	User interactions
Two-way binding	`bind:value={x}`	Form inputs
Lifecycle	`onMount(fn)` `onDestroy(fn)`	Setup/cleanup

Head elements	`<svelte:head>`
	`  <title>...</title>`
	`</svelte:head>`
		Per-page title/meta

Key TypeScript Syntax Explained in This Tutorial

Syntax	Meaning	Example
`x: string`	Type annotation	`let name: string = 'hi'`
`x?: string`	Optional property	`interface { href?: string }`
`A | B`	Union type (A or B)	`string | undefined`
`T[]`	Array of type T	`RegisteredWebhook[]`
`Map`	Map with key/value types	`Map<string, Metrics>`
`() => void`	Fn, no args, no return	`onconfirm: () => void`
`x?.y`	Optional chaining	`desc?.toLowerCase()`
`()`	Generic type parameter	`$state<Webhook[]>([])`
`interface`	Object shape definition	`interface Props { ... }`
`Snippet`	Renderable template type	`action?: Snippet`
`ReturnType`	Extract return type	`ReturnType<typeof setTimeout>`
`Record`	Object with typed keys	`Record<string, string>`
`as any`	Type assertion	`(err as any)?.message`

All code examples from the Sparrow webhook platform.
https://github.com/sarathsp06/sparrow